

Instructions for ACL-IJCNLP 2021 Proceedings

Anonymous ACL-IJCNLP submission

Abstract

This is the report of the project for the Web Recommender System course held at the University of Copenhagen.

1 Week 6 [Feb 3 - Feb 9]: Familiarize Yourself with the Datasets

1.1 Dataset Description and Import

The dataset used in this project is the “Musical Instruments” dataset from the Amazon Review data 2023 (5-core subset). The dataset consists of 14,000 item ratings from nearly 900 users on approximately 500 items. The data is split into training and test sets, with 80% of the data allocated for training (“train.tsv”) and 20% for testing (“test.tsv”).

The dataset was imported using the Python library `pandas` ([pandas development team, 2020](#)) and its `read_csv` function.

1.2 Data Cleaning and Preprocessing

To ensure the quality of the dataset, we performed the following preprocessing steps:

- Removal of missing values (entries with missing user ID, item ID, or rating).
- Deduplication by keeping the most recent rating when a user rated the same item multiple times.
- Ensuring that all users in the test set are also present in the training set.

In terms of dimensions the effect of the cleaning can be summarized in the [Table 1](#).

Dataset	Before Cleaning	After Cleaning
Train Set	11000	9913
Test Set	3000	1822

Table 1: Effect of the Data Cleaning Process

1.3 Exploratory Data Analysis (EDA) and Statistics

To gain insights into the dataset, we computed the following statistics:

- Distribution of ratings per user in [Figure 1](#)
- Distribution of ratings per item in [Figure 2](#)
- Overall rating value distribution in [Figure 3](#)

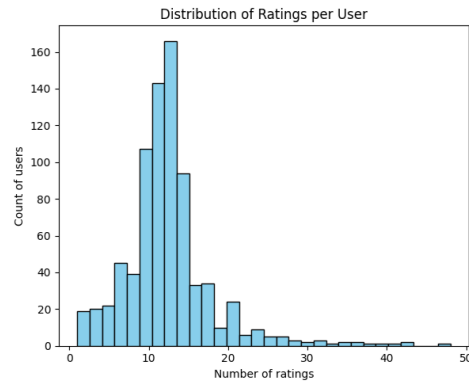


Figure 1: Distribution of Ratings per User

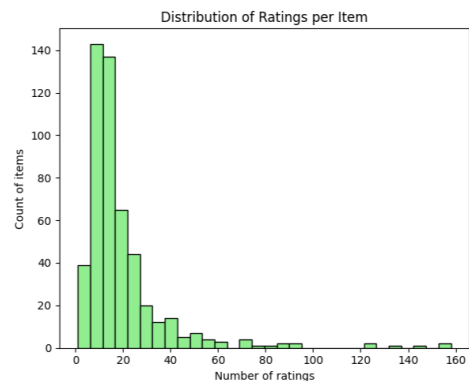


Figure 2: Distribution of Ratings per Item

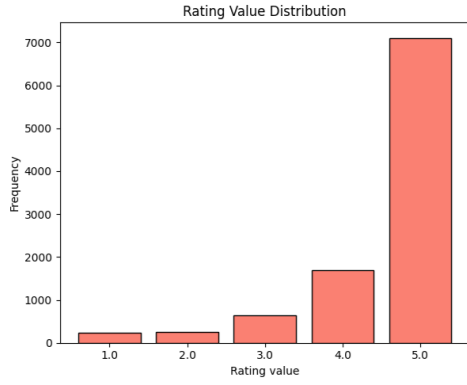


Figure 3: Distribution of Ratings per Vlaue

Also, to show the long-tail distribution of ratings, we plotted the number of ratings per item in Figure 4.

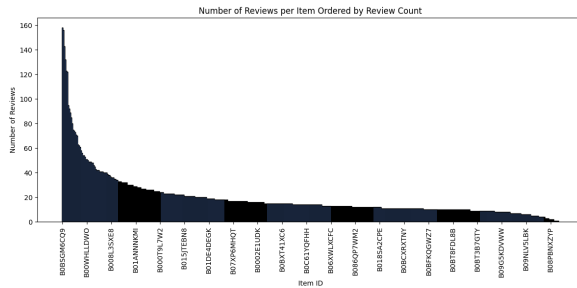


Figure 4: Long tail distribution of rating over all the items

1.4 Highly Rated Items

To identify popular items, we computed the number of ratings ≥ 3 for each item and reported the top 5 most highly rated items in Table 2.

Item ID	High Ratings Count	Average High Rating
B0BSGM6CQ9	153	4.784
B0BPJ4Q6FJ	153	4.804
B09857JRP2	141	4.809
B0BCK6L7S5	120	4.583
B0BTC9YJ2W	119	4.815

Table 2: Top 5 Most Highly Rated Items

1.5 Discussion

The dataset exhibits common characteristics of user-item interactions:

- There is a long-tail distribution where a small number of items receive most of the ratings, while many items have very few ratings.
- The majority of users have rated only a few items, which can lead to sparsity issues in collaborative filtering approaches.

- Certain items dominate the high-rated category, which may bias recommendation systems.

2 Week 7 [Feb 10 - Feb 16]: Collaborative Filtering Recommender System

2.1 Top Popular (TopPop) Recommender System

As a baseline, I first implemented the **Top Popular (TopPop) recommender system**. This model recommends the most popular items based on the number of “high” ratings (ratings ≥ 3) in the training set. I computed the frequency of high ratings per item and selected the top-k items as recommendations for all users.

While computationally inexpensive, this method lacks personalization since it does not consider individual user preferences.

2.2 Collaborative Filtering Models

I then used to Scikit-Surprise library to be able to compare it with one neighborhood-based model and one latent factor model. The ones I chose are:

- **User-based k-Nearest Neighbors with Means (KNNWithMeans)**: A neighborhood-based approach that identifies similar users based on their rating history.
- **Singular Value Decomposition (SVD)**: A latent factor model that decomposes the user-item interaction matrix into lower-dimensional representations.

2.3 Hyperparameter Tuning and Evaluation

To optimize both models and finde the best hyperparameters, I performed a Grid Search with **5-fold cross-validation** on the training set. The parameters I tuned included:

- **KNNWithMeans**
 - **k**: [5, 10, 20, 30, 40, 50, 70, 100]
 - **sim_options**:
 - * **name**: ['cosine', 'msd']
 - * **user_based**: [True]
- **SVD**
 - **n_factors**: [3, 5, 10, 20, 50, 100]
 - **n_epochs**: [5, 10, 20, 50, 100]

Model	Best Hyperparameters	Best RMSE
KNNWithMeans	'k': 70, 'sim_options': {'name': 'cosine', 'user_based': True}	0.9113
SVD	'n_factors': 10, 'n_epochs': 20	0.8445

Table 3: Best Hyperparameters and RMSE for Each Model

The evaluation metric chosen was **Root Mean Square Error (RMSE)**. RMSE penalizes larger errors more than smaller ones, making it a good measure to capture the overall predictive accuracy of our recommendation models. Lower RMSE indicates that the model predictions are closer to the actual ratings.

2.4 Results and Discussion

The best hyperparameters for each model can be found in [Table 3](#).

The SVD model, despite its low dimensional latent space, outperformed the KNNWithMeans model with a lower RMSE of 0.8445 compared to 0.9113. This indicates that the latent factor model is better at capturing the underlying patterns in the user-item interaction matrix.

After that both models have been trained again on the whole training set, to make sure that no data available for training is left behind before the evaluation on the test set.

References

The pandas development team. 2020. [pandas-dev/pandas: Pandas](#).